Status	Finished
Started	Monday, 10 August 2026, 4:07 PM
Completed	Monday, 10 August 2026, 4:22 PM
Duration	14 mins 38 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Given an integer N, check whether N the given number can be made a perfect square after adding 1 to it.

Input Format:

Single integer input.

Output Format:

Yes or No.

Example Input:

24

Output:

Yes

Example Input:

26

Output:

No

For example:

Input	Result
24	Yes

Answer: (penalty regime: 0 %)

```
import math

def check_perfect_square(n):
    target = n + 1
    if target < 0:
        return "No"
    root = int(math.isqrt(target))
    return "Yes" if root * root == target else "No"

# Example Usage
n = int(input())
print(check_perfect_square(n))

```

	Input	Expected	Got	
✓	24	Yes	Yes	✓
✓	26	No	No	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a program to calculate and print the Electricity bill where the unit consumed by the user is given from test case. It prints the total amount the customer has to pay. The charge are as follows:

Unit	Charge / Unit
Upto 199	@1.20
200 and above but less than 400	@1.50
400 and above but less than 600	@1.80
600 and above	@2.00

If bill exceeds Rs.400 then a surcharge of 15% will be charged and the minimum bill should be of Rs.100/-

Sample Test Cases

Test Case 1

Input

50

Output

100.00

Test Case 2

Input

300

Output

517.50

For example:

Input	Result
100.00	120.00
500	1035.00

Answer: (penalty regime: 0 %)

```
import sys

def calculate_electricity_bill():
    # Read input from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    units = float(input_data[0])

    # Determine the flat charge per unit based on brackets
    if units <= 199:
        rate = 1.20
    elif units < 400:
        rate = 1.50
    elif units < 600:
        rate = 1.80
    else:
        rate = 2.00

    # Calculate base bill
    total_amount = units * rate

    # Apply 15% surcharge if bill exceeds Rs. 400
    if total_amount > 400:
        total_amount += total_amount * 0.15

    # Enforce a minimum bill of Rs. 100
```

```
    if total_amount < 100:
        total_amount = 100.00

    # Print result formatted to 2 decimal places
    print(f"{total_amount:.2f}")

if __name__ == "__main__":
    calculate_electricity_bill()

```

	Input	Expected	Got	
✓	50	100.00	100.00	✓
✓	100.00	120.00	120.00	✓
✓	500	1035.00	1035.00	✓
✓	700	1610.00	1610.00	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Most years have 365 days. However, the time required for the Earth to orbit the Sun is actually slightly more than that. As a result, an extra day, February 29, is included in some years to correct for this difference. Such years are referred to as leap years. The rules for determining whether or not a year is a leap year follow:

• Any year that is divisible by 400 is a leap year.

• Of the remaining years, any year that is divisible by 100 is not a leap year.

• Of the remaining years, any year that is divisible by 4 is a leap year.

• All other years are not leap years.

Write a program that reads a year from the user and displays a message indicating whether or not it is a leap year.

Sample Input 1

1900

Sample Output 1

1900 is not a leap year.

Sample Input 2

2000

Sample Output 2

2000 is a leap year.

Answer: (penalty regime: 0 %)

```
import sys

def check_leap_year():
    # Read year from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    year = int(input_data[0])

    # Evaluate leap year logic rules
    if year % 400 == 0:
        is_leap = True
    elif year % 100 == 0:
        is_leap = False
    elif year % 4 == 0:
        is_leap = True
    else:
        is_leap = False

    # Format and print the exact required string output
    if is_leap:
        print(f"{year} is a leap year.")
    else:
        print(f"{year} is not a leap year.")

if __name__ == "__main__":
    check_leap_year()

```

	Input	Expected	Got	
✓	1900	1900 is not a leap year.	1900 is not a leap year.	✓
✓	2000	2000 is a leap year.	2000 is a leap year.	✓
✓	2100	2100 is not a leap year.	2100 is not a leap year.	✓
✓	2020	2020 is a leap year.	2020 is a leap year.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

A triangle can be classified based on the lengths of its sides as equilateral, isosceles or scalene. All three sides of an equilateral triangle have the same length. An isosceles triangle has two sides that are the same length, and a third side that is a different length. If all of the sides have different lengths then the triangle is scalene.

Write a program that reads the lengths of the three sides of a triangle from the user. Then display a message that states the triangle's type.

Sample Input 1

60

60

60

Sample Output 1

That's a equilateral triangle

Sample Input 2

40

40

80

Sample Output 2

That's a isosceles triangle

Sample Input 3

50

60

70

Sample Output 3

That's a scalene triangle

For example:

Input	Result
60 60 60	That's a equilateral triangle
40 40 80	That's a isosceles triangle

Answer: (penalty regime: 0 %)

```
import sys

def classify_triangle():
    # Read inputs from standard input
    input_data = sys.stdin.read().split()
    if len(input_data) < 3:
        return

    # Convert inputs to numbers to handle decimals or integers
    a = float(input_data[0])
    b = float(input_data[1])
    c = float(input_data[2])

    # Evaluate side lengths sequentially
    if a == b == c:
        print("That's a equilateral triangle")
    elif a == b or b == c or a == c:
        print("That's a isosceles triangle")
    else:
```

```
        print("That's a scalene triangle")

if __name__ == "__main__":
    classify_triangle()

```

	Input	Expected	Got	
✓	60 60 60	That's a equilateral triangle	That's a equilateral triangle	✓
✓	40 40 80	That's a isosceles triangle	That's a isosceles triangle	✓
✓	50 60 70	That's a scalene triangle	That's a scalene triangle	✓
✓	50 50 80	That's a isosceles triangle	That's a isosceles triangle	✓
✓	10 10 10	That's a equilateral triangle	That's a equilateral triangle	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a Python program that accepts three parameters. The first parameter is an integer. The second is one of the following mathematical operators: +, -, /, or *. The third parameter will also be an integer.

The function should perform a calculation and return the results. For example, if the function is passed 6 and 4, it should return 24.

Sample Input Format:

11

+

14

Sample Output Format:

25

Answer: (penalty regime: 0 %)

```
import sys

def calculate_result():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if len(input_data) < 3:
        return

    # Parse parameters sequentially
    num1 = int(input_data[0])
    operator = input_data[1]
    num2 = int(input_data[2])

    # Evaluate the mathematical operator
    if operator == '+':
        result = num1 + num2
    elif operator == '-':
        result = num1 - num2
    elif operator == '*':
        result = num1 * num2
    elif operator == '/':
        # Use integer division (//) or float division (/) depending on requirement
        # This handles division by zero safely
        if num2 == 0:
            print("Error: Division by zero")
            return
        result = num1 // num2  # Returns integer to match sample format
    else:
        print("Error: Invalid operator")
        return

    # Print the clean resulting calculation
    print(result)

if __name__ == "__main__":
    calculate_result()

```

	Input	Expected	Got	
✓	11 + 14	25	25	✓

	Input	Expected	Got	
✓	45 - 50	-5	-5	✓
✓	12 * 100	1200	1200	✓
✓	18 / 2	9.0	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a program that accepts 5 inputs and returns the count of how many of those 5 are odd.

For example,

If the five inputs are 12, 17, 19, 14, and 115, there are three odd numbers 17, 19 and 115. So, the program must return 3.

Similarly,

If the five inputs are 15, 0, -12, 19, and 28, there are two odd numbers 15 and 19. So, the program must return 2.

Observe that zero is considered an even number.

For example:

Input	Result
12 17 19 14 115	3
15 0 -12 19 28	2

Answer: (penalty regime: 0 %)

```
import sys

def count_odd_numbers():
    # Read all inputs from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    # Map the first 5 values to integers
    numbers = [int(x) for x in input_data[:5]]

    # Count how many numbers are odd (remainder when divided by 2 is not 0)
    odd_count = sum(1 for num in numbers if num % 2 != 0)

    # Output the final count
    print(odd_count)

if __name__ == "__main__":
    count_odd_numbers()

```

	Input	Expected	Got	
✓	12 17 19 14 115	3	3	✓
✓	15 0 -12 19 28	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

The Chinese zodiac assigns animals to years in a 12 year cycle. One 12 year cycle is shown in the table below. The pattern repeats from there, with 2012 being another year of the dragon, and 1999 being another year of the hare.

Year Animal

2000 Dragon

2001 Snake

2002 Horse

2003 Sheep

2004 Monkey

2005 Rooster

2006 Dog

2007 Pig

2008 Rat

2009 Ox

2010 Tiger

2011 Hare

Write a program that reads a year from the user and displays the animal associated with that year. Your program should work correctly for any year greater than or equal to zero, not just the ones listed in the table.

Sample Input 1

2010

Sample Output 1

2010 is the year of the Tiger.

Sample Input 2

2020

Sample Output 2

2020 is the year of the Rat.

Answer: (penalty regime: 0 %)

```
import sys

def get_chinese_zodiac():
    # Read year from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    year = int(input_data[0])

    # Ordered array matching the index offset from the year 2000
    animals = [
        "Dragon", "Snake", "Horse", "Sheep",
        "Monkey", "Rooster", "Dog", "Pig",
        "Rat", "Ox", "Tiger", "Hare"
    ]

    # Calculate index position relative to base year 2000
    index = (year - 2000) % 12
    animal = animals[index]

    # Print the exact matching sample output statement
    print(f"{year} is the year of the {animal}.")

if __name__ == "__main__":
    get_chinese_zodiac()

```

	Input	Expected	Got	
✓	2010	2010 is the year of the Tiger.	2010 is the year of the Tiger.	✓
✓	2020	2020 is the year of the Rat.	2020 is the year of the Rat.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

IN / OUT

Ms. Sita, the faculty handling programming lab for you is very strict. Your seniors have told you that she will not allow you to enter the week's lab if you have not completed atleast half the number of problems given last week. Many of you didn't understand this statement and so they requested the good programmers from your batch to write a program to find whether a student will be allowed into a week's lab given the number of problems given last week and the number of problems solved by the student in that week.

Input Format:

Input consists of 2 integers.

The first integer corresponds to the number of problems given and the second integer corresponds to the number of problems solved.

Output Format:

Output consists of the string "IN" or "OUT".

Sample Input and Output:

Input

8

3

Output

OUT

For example:

Input	Result
8 3	OUT

Answer: (penalty regime: 0 %)

```
import sys

def check_lab_entry():
    # Read inputs from standard input
    input_data = sys.stdin.read().split()
    if len(input_data) < 2:
        return

    total_problems = int(input_data[0])
    solved_problems = int(input_data[1])

    # Check if solved problems meet or exceed half of total problems
    if solved_problems >= (total_problems / 2.0):
        print("IN")
    else:
        print("OUT")

if __name__ == "__main__":
    check_lab_entry()
```

	Input	Expected	Got	
✓	8 3	OUT	OUT	✓

	Input	Expected	Got	
✓	8 5	IN	IN	✓
✓	20 9	OUT	OUT	✓
✓	50 31	IN	IN	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

A certain type of steel is used to test and give grade according to the following conditions.

1. Hardness of the steel must be greater than 50
2. Carbon content of the steel must be less than 0.7
3. Tensile strength must be greater than 5600

The grades awarded are as follows:

- Grade is 10 if all three conditions are met
- Grade is 9 if conditions (1) and (2) are met
- Grade is 8 if conditions (2) and (3) are met
- Grade is 7 if conditions (1) and (3) are met
- Grade is 6 if only one condition is met
- Grade is 5 if none of the three conditions are met

Write a program to display the grade of the steel, based on the values of hardness, carbon content and tensile strength of the steel, given by the user.

Input

53

0.6

5602

Output:

10

Answer: (penalty regime: 0 %)

```
import sys

def grade_steel():
    # Read all input values from standard input
    input_data = sys.stdin.read().split()
    if len(input_data) < 3:
        return

    hardness = float(input_data[0])
    carbon = float(input_data[1])
    tensile = float(input_data[2])

    # Evaluate the status of each independent rule constraint
    c1 = hardness > 50
    c2 = carbon < 0.7
    c3 = tensile > 5600

    # Assign specific grade tiers based on rules
    if c1 and c2 and c3:
        grade = 10
    elif c1 and c2:
        grade = 9
    elif c2 and c3:
        grade = 8
    elif c1 and c3:
        grade = 7
    elif c1 or c2 or c3:
        grade = 6
    else:
        grade = 5

    print(grade)
```

```
34 if __name__ == "__main__":
35     grade_steel()
36
```

	Input	Expected	Got	
✓	53 0.6 5602	10	10	✓
✓	45 0 4500	6	6	✓

Passed all tests! ✓

Correct

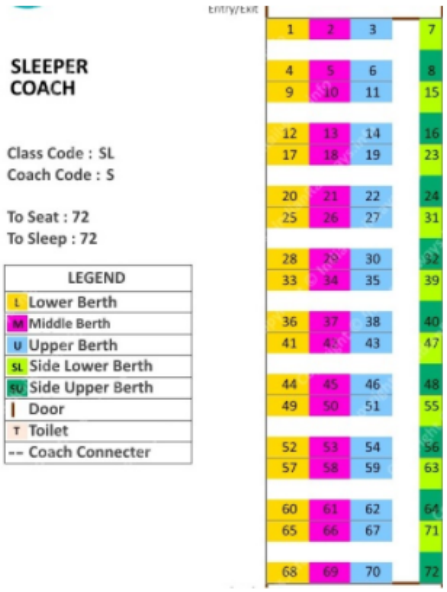
Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Write a program to determine the type of berth when the seat / berth number in the train is given.



Input Format:

Input consists of a single integer. Assume that the range of input is between 1 and 72.

Output Format:

Output consists of a single string. [Upper or Middle or Lower or Side Lower or Side Upper]

Sample Input 1:

9

Sample Output 1:

Lower Berth

Answer: (penalty regime: 0 %)

```
import sys

def determine_berth():
    # Read seat number from standard input
    input_data = sys.stdin.read().split()
    if not input_data:
        return

    seat = int(input_data[0])

    # Check compartment positioning via a modulo 8 operation
    remainder = seat % 8

    if remainder == 1 or remainder == 4:
        print("Lower Berth")
    elif remainder == 2 or remainder == 5:
        print("Middle Berth")
    elif remainder == 3 or remainder == 6:
        print("Upper Berth")
    elif remainder == 7:
        print("Side Lower Berth")
    elif remainder == 0:
        print("Side Upper Berth")

if __name__ == "__main__":
    determine_berth()

```

	Input	Expected	Got	
✓	9	Lower Berth	Lower Berth	✓
✓	72	Side Upper Berth	Side Upper Berth	✓
✓	20	Lower Berth	Lower Berth	✓
✓	34	Middle Berth	Middle Berth	✓
✓	39	Side Lower Berth	Side Lower Berth	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.